

// Jivops

Consul

Guía completa — 30 lecciones

Fundamentos · Service discovery · KV y configuración · Connect/service mesh · Multi-datacenter · Producción avanzada

Contenido

Módulo 1: Fundamentos de Consul

- Lección 01 — Qué es Consul y los problemas que resuelve (discovery, config, segmentation)
- Lección 02 — Arquitectura de Consul — servers y agents
- Lección 03 — El protocolo Gossip y Raft en Consul
- Lección 04 — Instalación de un cluster de Consul
- Lección 05 — Consul UI y CLI

Módulo 2: Service discovery y health checking

- Lección 06 — Registro de servicios en el catálogo de Consul
- Lección 07 — Health checks — tipos y configuración
- Lección 08 — DNS interface de Consul
- Lección 09 — Consul Connect vs Consul API para descubrimiento
- Lección 10 — Multi-datacenter service discovery

Módulo 3: Consul KV y gestión de configuración

- Lección 11 — El almacén clave-valor de Consul
- Lección 12 — Watches — reaccionar a cambios de configuración
- Lección 13 — Consul Template — generar archivos de configuración dinámicamente
- Lección 14 — Sesiones y locks distribuidos con Consul
- Lección 15 — Prepared queries

Módulo 4: Consul Connect — service mesh

- Lección 16 — Qué es Consul Connect y el modelo de sidecar proxy
- Lección 17 — mTLS automático entre servicios
- Lección 18 — Intentions — autorización de servicio a servicio
- Lección 19 — Integración de Connect con Envoy
- Lección 20 — Connect en Kubernetes vs Connect en Nomad/VMs

Módulo 5: Multi-datacenter y federación

- Lección 21 — WAN gossip y federación entre datacenters
- Lección 22 — Replicación de Consul KV entre datacenters
- Lección 23 — Mesh gateways para tráfico entre datacenters
- Lección 24 — Consul Enterprise vs Consul OSS — diferencias clave
- Lección 25 — Admin partitions y namespaces

Módulo 6: Producción avanzada

Lección 26 — ACLs de Consul — seguridad de acceso

Lección 27 — Backup y disaster recovery de Consul

Lección 28 — Monitoreo de Consul con Prometheus

Lección 29 — Rendimiento y tuning de un cluster de Consul

Lección 30 — Proyecto final: plataforma Consul completa lista para producción

Módulo 1: Fundamentos de Consul

Lección 01 — Qué es Consul y los problemas que resuelve (discovery, config, segmentation)

Descubrimiento de servicios (saber dónde está cada servicio y si está saludable), gestión centralizada de configuración distribuida, y segmentación de red (controlar qué servicios pueden comunicarse entre sí), combinando esas capacidades en una única herramienta.

Usar herramientas completamente separadas para descubrimiento de servicios, gestión de configuración y segmentación de red requeriría que el equipo aprenda, mantenga e integre manualmente múltiples sistemas independientes entre sí; consolidar esas capacidades relacionadas en una única herramienta como Consul reduce esa fragmentación, simplificando la operación general de una infraestructura distribuida con decenas de microservicios distintos.

Lección 02 — Arquitectura de Consul — servers y agents

Los servers forman el núcleo que mantiene el estado consistente del cluster (el catálogo de servicios, la configuración) mediante consenso entre ellos, mientras los agents corren en cada nodo donde viven los servicios reales, registrando esos servicios localmente y reportando información hacia los servers.

Con un agent corriendo localmente en el mismo nodo, cada aplicación solo necesita comunicarse con ese punto de contacto cercano (normalmente en localhost) para registrarse, sin necesitar conocer ni gestionar directamente la ubicación de los servers centrales, que podrían estar en otros nodos distintos; esa simplicidad de interactuar con un agent local reduce la complejidad que cada aplicación individual tendría que manejar para participar correctamente en el cluster de Consul.

Lección 03 — El protocolo Gossip y Raft en Consul

Gossip se usa para que los miembros del cluster (servers y agents) descubran y mantengan conocimiento de quién más está presente y activo de forma eficiente y descentralizada, mientras Raft es el algoritmo de consenso específico que usan los servers entre sí para mantener acordado y consistente el estado autoritativo del cluster.

El estado autoritativo del cluster necesita una fuente de verdad confiable y consistente, algo que se logra mejor con un grupo reducido y bien coordinado de participantes (los servers) usando un mecanismo formal y riguroso como Raft; incluir a todos los agents (potencialmente muchos más numerosos) en ese mismo consenso formal sería innecesariamente costoso y complejo, razón por la cual Gossip (más ligero) se usa para el descubrimiento general de membresía entre todos los miembros, mientras Raft se reserva específicamente para el consenso formal entre servers.

Lección 04 — Instalación de un cluster de Consul

Se recomienda un número impar de servers (típicamente 3 o 5) para que el algoritmo de consenso Raft pueda alcanzar mayoría incluso si uno o más servers fallan, determinando cuántas fallas simultáneas puede tolerar el cluster sin perder la capacidad de tomar decisiones.

Con 3 servers, incluso si uno falla, los 2 restantes todavía representan una mayoría suficiente (más de la mitad) para que el algoritmo de consenso Raft siga funcionando y tomando decisiones correctamente; con un único server, esa misma falla eliminaría por completo la única fuente de consenso disponible, dejando al cluster completamente sin capacidad de operar hasta que ese único server se recupere.

Lección 05 — Consul UI y CLI

La UI ofrece una vista visual del catálogo de servicios, su estado de salud, y el almacén KV, facilitando la exploración y el diagnóstico visual rápido, mientras la CLI permite automatizar tareas repetitivas e integrarse con scripts, siendo la forma preferida de interactuar con Consul desde pipelines automatizados.

Un script automatizado de despliegue necesita ejecutar el registro del nuevo servicio sin ninguna intervención manual humana en cada ejecución del flujo; la interfaz web requeriría que una persona interactúe físicamente haciendo clic, algo incompatible con un proceso completamente automatizado que se repite en cada despliegue, mientras la CLI puede invocarse directamente desde el script como cualquier otro comando programático, sin esa necesidad de interacción humana.

Módulo 2: Service discovery y health checking

Lección 06 — Registro de servicios en el catálogo de Consul

Su dirección IP y puerto reales, etiquetas (tags) opcionales que describen características adicionales del servicio, metadatos personalizados, y opcionalmente definiciones de health checks que determinan si ese servicio se considera actualmente saludable.

Sin un health check asociado, el catálogo podría seguir devolviendo la dirección de una instancia que técnicamente fue registrada pero que ya dejó de funcionar correctamente (por ejemplo, tras una falla); el health check verifica activamente que esa instancia específica sigue respondiendo y funcionando, siendo precisamente lo que permite que otros servicios confíen en que la dirección devuelta por el catálogo corresponde realmente a algo disponible, no solo a algo que fue registrado en algún momento pasado.

Lección 07 — Health checks — tipos y configuración

Un health check HTTP es verificado activamente por Consul, que hace peticiones periódicas hacia un endpoint específico del servicio para confirmar su estado, mientras un health check TTL es pasivo, requiriendo que el propio servicio reporte activamente su estado a Consul antes de que expire ese tiempo límite, o se marcará como no saludable.

Un servicio de trabajo en lotes podría no tener expuesto ningún endpoint HTTP accesible específicamente para verificación externa, haciendo que un health check de tipo HTTP no tenga forma real de consultar su estado activamente; con TTL, ese mismo servicio puede reportar directamente su propio estado hacia Consul desde dentro de su propia lógica (por ejemplo, confirmando que completó su última tarea correctamente), sin necesitar exponer ningún endpoint adicional solo para ese propósito de verificación.

Lección 08 — DNS interface de Consul

Permite que cualquier aplicación compatible con resolución DNS estándar (prácticamente cualquier software) pueda descubrir la dirección de un servicio registrado en Consul simplemente haciendo una consulta DNS normal, sin necesitar implementar ninguna integración específica con la API de Consul dentro de su propio código.

Una aplicación legacy ya sabe hacer consultas DNS estándar, ya que esa es una capacidad básica y universal de prácticamente cualquier software de red, sin importar cuándo fue escrita; como la interfaz DNS de Consul se presenta como un servidor DNS normal y compatible, esa aplicación legacy puede seguir usando exactamente el mismo mecanismo de resolución DNS que ya conocía, sin necesitar ninguna modificación de su código original para aprovechar el descubrimiento de servicios que Consul ofrece.

Lección 09 — Consul Connect vs Consul API para descubrimiento

Con la API directa, la aplicación misma consulta activamente el catálogo y gestiona la lógica de conexión hacia el servicio destino, mientras con Connect, un proxy sidecar maneja esa lógica de descubrimiento y conexión de forma transparente, añadiendo también capacidades adicionales como cifrado automático del tráfico entre servicios.

El desarrollador de cada aplicación puede enfocarse exclusivamente en la lógica de negocio real, sin tener que implementar manualmente y de forma correcta la lógica de consultar el catálogo, manejar reintentos, y gestionar la conexión hacia el servicio destino; delegar esa responsabilidad hacia el proxy sidecar (una capa de infraestructura compartida) simplifica significativamente el código de cada aplicación individual, evitando que cada equipo tenga que reimplementar esa misma funcionalidad por separado.

Lección 10 — Multi-datacenter service discovery

Permite consultar servicios registrados en un datacenter remoto distinto desde un datacenter local, posibilitando que una aplicación en un datacenter encuentre y se conecte con un servicio que vive físicamente en otro datacenter distinto, sin necesitar configuración manual separada para cada combinación posible.

Sin esa federación automática, cada aplicación en un datacenter tendría que conocer de antemano y configurar manualmente cómo contactar servicios específicos del otro datacenter distante; con la federación de Consul, una aplicación en el datacenter europeo puede simplemente consultar y encontrar servicios registrados en el datacenter americano (o viceversa) sin ese trabajo manual previo, automatizando significativamente el descubrimiento entre operaciones distribuidas geográficamente.

Módulo 3: Consul KV y gestión de configuración

Lección 11 — El almacén clave-valor de Consul

Provee un espacio centralizado y distribuido para almacenar configuración dinámica (como feature flags, parámetros de aplicación, o valores que varían según el entorno), accesible por múltiples aplicaciones distintas de forma consistente a través del cluster.

Si el feature flag estuviera hardcodeado directamente en el código, cambiar su valor requeriría modificar ese código y volver a desplegar la aplicación completa; con el valor almacenado en el KV store centralizado de Consul, basta con modificarlo directamente ahí, y cualquier aplicación que lo consulte reflejará automáticamente ese cambio la próxima vez que lo lea, sin necesitar ningún redespigüe de código real.

Lección 12 — Watches — reaccionar a cambios de configuración

Permite que una aplicación (o un script) sea notificada automáticamente tan pronto como el valor de esa clave específica cambie, disparando una acción definida (como recargar configuración) en vez de tener que consultar repetidamente esa clave de forma manual para detectar si cambió.

Un script consultando repetidamente (polling) esa misma clave cada pocos segundos generaría carga innecesaria la mayoría de las veces, cuando el valor en realidad no ha cambiado todavía; un watch, en cambio, notifica únicamente cuando realmente ocurre un cambio real en esa clave específica, evitando todas esas consultas innecesarias y reaccionando de forma mucho más eficiente solo ante cambios genuinos, en vez de desperdiciar recursos revisando constantemente algo que la mayor parte del tiempo permanece igual.

Lección 13 — Consul Template — generar archivos de configuración dinámicamente

Genera y actualiza automáticamente un archivo de configuración real (como un archivo de configuración de Nginx o una aplicación) basándose en una plantilla y en datos obtenidos dinámicamente de Consul, regenerando ese archivo y opcionalmente ejecutando una acción (como recargar el servicio) cada vez que los datos relevantes cambian.

Si las instancias de un servicio cambian frecuentemente (escalando hacia arriba o recuperándose de una falla), mantener actualizado manualmente ese archivo de configuración cada vez sería un proceso propenso a olvido o retraso humano real; Consul Template detecta esos cambios automáticamente y regenera el archivo correspondiente sin depender de que alguien recuerde hacerlo manualmente, mejorando significativamente la confiabilidad al eliminar ese riesgo real de que el archivo quede desactualizado.

Lección 14 — Sesiones y locks distribuidos con Consul

Garantiza que, aunque múltiples instancias intenten adquirir el mismo lock simultáneamente, solo una de ellas lo consiga exitosamente en un momento dado, evitando que la misma tarea crítica se ejecute de forma duplicada y potencialmente conflictiva por varias instancias al mismo tiempo.

Sin ningún mecanismo de coordinación, dos instancias distintas podrían ejecutar simultáneamente exactamente la misma tarea crítica (por ejemplo, un proceso de facturación programado) sin que ninguna de las dos sepa que la otra también lo está haciendo al mismo tiempo, potencialmente generando resultados duplicados o efectos secundarios conflictivos reales dependiendo de la naturaleza de esa tarea;

el lock distribuido es precisamente el mecanismo que previene ese escenario, garantizando que solo una instancia ejecute la tarea en cada momento.

Lección 15 — Prepared queries

Permiten definir una consulta reutilizable con lógica adicional, como una estrategia de failover automático hacia un datacenter alternativo si el servicio local no tiene ninguna instancia saludable disponible en ese momento, sin que el cliente que hace la consulta necesite implementar esa lógica de failover por su cuenta.

Si cada aplicación cliente tuviera que implementar su propia lógica de failover hacia otro datacenter (detectar que no hay instancias saludables localmente, saber hacia qué datacenter alternativo intentar), esa complejidad se duplicaría innecesariamente en cada aplicación distinta; centralizar esa lógica dentro de una prepared query permite que cualquier cliente simplemente haga la consulta normal, sin necesitar replicar esa complejidad de failover dentro de su propio código individual.

Módulo 4: Consul Connect — service mesh

Lección 16 — Qué es Consul Connect y el modelo de sidecar proxy

Inyecta un proxy auxiliar (sidecar) junto a cada instancia de servicio, de forma que todo el tráfico entrante y saliente de ese servicio pase a través de ese proxy, que gestiona de forma transparente aspectos como el descubrimiento, el cifrado y la autorización, sin que la aplicación misma necesite implementar esa lógica directamente.

El desarrollador de cada nuevo servicio puede enfocarse exclusivamente en la lógica de negocio real, sin tener que implementar manualmente y de forma correcta el cifrado y la autorización dentro de su propio código; delegar esa responsabilidad hacia el proxy sidecar (una capa de infraestructura compartida y ya probada) simplifica significativamente el desarrollo, además de garantizar consistencia de seguridad en todo el mesh sin depender de que cada equipo lo implemente correctamente por separado.

Lección 17 — mTLS automático entre servicios

Garantiza que ambos extremos de la comunicación (no solo uno) verifiquen mutuamente la identidad del otro mediante certificados, además de cifrar todo el tráfico entre ellos, sin que ningún desarrollador tenga que gestionar manualmente esos certificados o su rotación periódica.

Con TLS tradicional, el cliente verifica que el servidor es quien dice ser, pero el servidor no tiene forma real de verificar la identidad del cliente que se está conectando, permitiendo potencialmente que cualquier cliente no autorizado intente conectarse; mTLS añade esa verificación adicional en la dirección opuesta, confirmando también la identidad del cliente antes de aceptar la conexión, ofreciendo una seguridad más robusta al verificar ambos extremos de la comunicación en vez de confiar solo en una dirección de esa verificación.

Lección 18 — Intentions — autorización de servicio a servicio

Definen explícitamente qué servicios tienen permiso de comunicarse con cuáles otros servicios dentro del mesh, actuando como una capa de autorización que los proxies sidecar consultan y aplican antes de permitir que el tráfico entre dos servicios específicos realmente fluya.

Sin esa restricción explícita mediante Intentions, cualquier servicio dentro del mesh (incluido uno comprometido por un atacante) podría intentar comunicarse libremente hacia el servicio sensible de base de datos de pagos; al definir explícitamente que solo el servicio de facturación autorizado tiene ese permiso específico, un atacante que comprometiera un servicio distinto no autorizado se encontraría con esa comunicación bloqueada, limitando el alcance real del daño que podría causar dentro del mesh completo.

Lección 19 — Integración de Connect con Envoy

Envoy es un proxy de datos de alto rendimiento, ampliamente adoptado en el ecosistema de service mesh, que Consul Connect configura y gestiona automáticamente como el componente real que intercepta y procesa el tráfico de red para cada servicio dentro del mesh.

Configurar manualmente un proxy como Envoy desde cero requeriría un conocimiento técnico específico y detallado que un equipo sin experiencia previa en Envoy probablemente no tendría; Consul abstrae esa complejidad, generando automáticamente la configuración correcta de Envoy basándose en las Intentions y la configuración general del mesh, permitiendo que ese equipo se beneficie de las capacidades del service mesh sin necesitar aprender esos detalles internos específicos de configuración.

Lección 20 — Connect en Kubernetes vs Connect en Nomad/VMs

En Kubernetes, Consul puede aprovechar un mecanismo de inyección automática de sidecars basado en anotaciones de Pod, simplificando el proceso de añadir el proxy a cada servicio, mientras en Nomad o VMs tradicionales ese proceso de incorporar el sidecar típicamente requiere una configuración más explícita dentro de la especificación del job o del despliegue correspondiente.

Un equipo que ya despliega sus aplicaciones usando manifiestos estándar de Kubernetes puede simplemente añadir una anotación específica a esos manifiestos ya existentes para que Consul inyecte automáticamente el sidecar correspondiente, en vez de tener que aprender y aplicar un proceso de configuración manual más extenso y distinto; esa simplicidad de adopción, integrada naturalmente dentro del flujo de trabajo ya familiar del equipo, reduce significativamente la barrera para empezar a usar Consul Connect.

Módulo 5: Multi-datacenter y federación

Lección 21 — WAN gossip y federación entre datacenters

El LAN gossip coordina la membresía entre agents y servers dentro de un mismo datacenter con baja latencia esperada, mientras el WAN gossip coordina específicamente entre los servers de distintos datacenters remotos, diseñado para tolerar la latencia y menor confiabilidad típica de conexiones entre ubicaciones geográficas distintas.

La conexión entre datacenters ubicados en distintas ciudades o países típicamente atraviesa internet público, con una latencia mucho mayor y una confiabilidad potencialmente menor que la red interna de un mismo datacenter local; el WAN gossip está diseñado específicamente considerando esas condiciones de red menos ideales, mientras el LAN gossip puede asumir la latencia mucho más baja y la confiabilidad más alta típica de una red local dentro del mismo datacenter.

Lección 22 — Replicación de Consul KV entre datacenters

Por defecto, el KV store de Consul es local a cada datacenter (no se replica automáticamente entre datacenters federados), lo que significa que un valor escrito en un datacenter no aparece automáticamente en otro, requiriendo una estrategia explícita adicional si se necesita esa sincronización entre datacenters.

Si se asumiera erróneamente que el KV store se replica automáticamente, se esperaría que un valor escrito en el datacenter local también esté disponible en el datacenter remoto; pero como el comportamiento real por defecto es que cada datacenter mantiene su propio KV store local sin sincronización automática, la aplicación en el datacenter remoto simplemente no encontraría ese valor, ya que nunca fue realmente escrito ahí, causando un problema de configuración faltante inesperado si esa suposición incorrecta no se verificó de antemano.

Lección 23 — Mesh gateways para tráfico entre datacenters

Actúan como un punto de entrada y salida centralizado para el tráfico del mesh que cruza entre datacenters, evitando que cada servicio individual necesite conectividad de red directa hacia servicios de otros datacenters, y permitiendo mantener el cifrado mTLS de extremo a extremo incluso a través de esa frontera entre datacenters.

Si cada servicio individual necesitara conectividad de red directa hacia servicios específicos de otros datacenters, la cantidad de conexiones distintas que configurar y mantener (y las reglas de firewall correspondientes) crecería significativamente con el número de servicios; centralizar ese tráfico a través de un único mesh gateway por datacenter reduce drásticamente esa complejidad, concentrando la conectividad entre datacenters en un punto manejable en vez de muchas conexiones individuales dispersas.

Lección 24 — Consul Enterprise vs Consul OSS — diferencias clave

Capacidades adicionales orientadas a organizaciones más grandes y con requisitos más complejos, como control de acceso más granular (namespaces, admin partitions), replicación avanzada entre regiones, y soporte comercial oficial, sin estar disponibles en la versión OSS gratuita.

Sin namespaces, todos los equipos de la organización compartirían el mismo espacio plano de servicios y configuración dentro del cluster, sin ninguna separación lógica real entre lo que pertenece a cada equipo; la capacidad de namespaces de Consul Enterprise permite segmentar lógicamente ese cluster compartido, dando a cada equipo su propio espacio aislado dentro de la misma infraestructura física, un beneficio real

específicamente valioso para una organización grande con múltiples equipos distintos.

Lección 25 — Admin partitions y namespaces

Un namespace provee aislamiento lógico dentro del mismo plano de control de Consul (compartiendo los mismos servers), mientras una admin partition ofrece un nivel de aislamiento más amplio, permitiendo incluso agents en redes completamente separadas, útil para escenarios donde se necesita una separación administrativa más fuerte entre distintas unidades organizacionales.

Dos unidades de negocio con redes completamente separadas y equipos de infraestructura independientes necesitarían un nivel de aislamiento más fuerte del que un namespace (que comparte el mismo plano de control y los mismos servers) podría ofrecer adecuadamente; las admin partitions permiten ese aislamiento más robusto, incluyendo agents en redes completamente separadas, ajustándose mejor a esa necesidad real de separación administrativa fuerte entre unidades organizacionales genuinamente independientes entre sí.

Módulo 6: Producción avanzada

Lección 26 — ACLs de Consul — seguridad de acceso

Define qué tokens o identidades tienen permiso para realizar qué operaciones específicas (como leer o escribir en el KV store, registrar servicios, o gestionar Intentions), aplicando el principio de menor privilegio para evitar que cualquiera con acceso a la red del cluster pueda realizar cualquier operación sin restricción.

Sin ACLs habilitadas, cualquier persona con conectividad de red hacia el cluster podría potencialmente leer o modificar valores sensibles del KV store (como credenciales o configuración crítica) sin ninguna verificación de si realmente tiene autorización para hacerlo; eso representaría un riesgo real tanto de errores accidentales como de acciones maliciosas deliberadas, siendo precisamente el escenario que las ACLs están diseñadas para prevenir.

Lección 27 — Backup y disaster recovery de Consul

Si el estado del cluster se corrompiera o se perdiera por completo (por ejemplo, una falla grave que afecte a la mayoría de los servers), sin un backup disponible habría que reconstruir manualmente toda la configuración, los valores del KV store y las reglas de Intentions desde cero, un proceso lento que extendería significativamente el tiempo de recuperación ante un desastre real.

Las Intentions controlan específicamente qué servicios tienen permiso de comunicarse entre sí dentro del service mesh; perder esas reglas sin backup significaría tener que reconstruir esa configuración de seguridad crítica desde cero, bajo la presión de una recuperación de desastre, aumentando el riesgo real de cometer errores que dejen el mesh temporalmente más expuesto (permitiendo comunicación no autorizada) o incorrectamente bloqueado, más allá de la simple interrupción operativa del servicio.

Lección 28 — Monitoreo de Consul con Prometheus

Métricas sobre la salud del consenso Raft entre servers (como la presencia de un líder activo), la latencia de las consultas al catálogo o al KV store, y el número de health checks fallando, dando visibilidad real de si Consul está funcionando eficientemente más allá de solo confirmar que el proceso está técnicamente activo.

Los procesos de Consul podrían estar técnicamente en ejecución mientras el grupo de servers, por alguna razón, no logra elegir un líder activo para el consenso Raft (un problema real que impide tomar decisiones o aceptar escrituras en el cluster); esa condición crítica no se reflejaría en una simple verificación de que el proceso está corriendo, razón por la cual monitorear específicamente el estado del consenso (presencia de líder) es necesario para detectar ese tipo de problema operativo grave.

Lección 29 — Rendimiento y tuning de un cluster de Consul

A medida que el número de servicios registrados y la frecuencia de consultas (watches, health checks) crece significativamente, la carga sobre los servers (que mantienen el estado consistente mediante Raft) también crece, pudiendo requerir ajustes como dimensionar mejor el hardware de los servers o distribuir mejor la carga de consultas.

Con 100 servicios, la carga de trabajo sobre los servers (mantener consistencia vía Raft, procesar health checks, responder consultas) es mucho menor que con 10,000 servicios distintos; si los servers originalmente dimensionados para esa carga menor no se ajustan en consecuencia a medida que el cluster crece significativamente, empezarían a mostrar signos reales de degradación de rendimiento, ya que su capacidad de hardware original podría no ser suficiente para ese volumen mucho mayor de trabajo.

Lección 30 — Proyecto final: plataforma Consul completa lista para producción

Cluster con servers en número impar para alta disponibilidad + service discovery con health checks apropiados + KV store con watches para configuración dinámica + Consul Connect con mTLS e Intentions configuradas + ACLs habilitadas + monitoreo con Prometheus y backup robusto del estado del cluster.

Intenta mencionar al menos dos pilares: alta disponibilidad de servers, service discovery con health checks, Connect con mTLS/Intentions o ACLs con monitoreo y backup.